

Tóm tắt kiến trúc và luồng dữ liệu

Hệ thống RAG y tế hướng dẫn chẩn đoán kết hợp hai kỹ thuật: *Hybrid Retrieval* (BM25 + embedding) từ paper NICE RAG ¹ ² và *RAPTOR Tree Retrieval* (cây phân cấp với collapsed-tree search) ³ ⁴. Kiến trúc tổng thể gồm ba giai đoạn chính: (1) **Tiền xử lý dữ liệu** – chuyển PDF/XML hướng dẫn sang Markdown, chia nhỏ văn bản theo mục/lục, tạo các “đoạn” (chunk) có kích thước tối ưu (split nếu >600 token, merge nếu <200, chồng lấp 50 token) ⁵. (2) **Xây dựng cây RAPTOR** – mã hóa các chunk bằng embedding (ví dụ SBERT), *phân cụm* (GMM) và *tóm tắt* (gọi LLM, ví dụ GPT-3.5) các cụm, tạo nút cây ở nhiều cấp độ trừu tượng ⁶ ⁷. Nút lá chứa chi tiết, các lớp cao hơn chứa summary của cluster. (3) **Triển khai chỉ mục kết hợp** – đánh chỉ mục từng nút lá và nút summary với BM25 và các vector embedding. Lúc truy vấn, hệ thống thực hiện **Hybrid Retrieval**: tra cứu BM25 và dense (cosine) trên tất cả node, kết hợp kết quả bằng công thức **Weighted Reciprocal Rank Fusion (WRRF)** ², sau đó **Rerank** các top-k context bằng Cross-Encoder cho độ chính xác cao. Cuối cùng, ngữ cảnh chọn lọc được đưa vào LLM (ví dụ GPT-4 hoặc GPT-4-mini) với prompt đặc biệt cho chẩn đoán phân biệt.

flowchart TD

subgraph Xử lý Dữ liệu

A[Hướng dẫn (PDF/XML)] --> B[Chuyển sang Markdown]

B --> C[Chia đoạn theo mục/lục]

C --> D{Kích thước đoạn}

D -- lớn (>600) --> E[Chia thêm (với overlap 50)]

D -- nhỏ (<200) --> F[Nối với đoạn lân cận]

D -- vừa phải --> G[Giữ nguyên]

E & F & G --> H[Chunks gốc (nút lá)]

end

subgraph Xây dựng Cây RAPTOR

H --> I[Mã hóa (SBERT embeddings)]

I --> J[Áp dụng UMAP + GMM (tự chọn số cụm bằng BIC)]

J --> K[Sử dụng LLM (GPT-3.5) tóm tắt từng cụm]

K --> L[Nút cha mới chứa summary]

L --> M[Tiếp tục đệ quy embedding + clustering cho node summary]

M --> N[Cây RAPTOR]

end

subgraph Đánh chỉ mục & Truy vấn

N --> O[Chỉ mục kết hợp:

- BM25 trên văn bản\n- Vector embeddings (SBERT) cho mỗi node]

P(User Query) --> Q[Xử lý query (chuẩn hóa triệu chứng)]

Q --> R{Hybrid Retrieval}

R --> S[BM25 search (keyword)]

R --> T[Vector search (cosine SBERT)]

S & T --> U[Weighted RRF kết hợp kết quả]

```

    U --> V[Top-k node candidates]
    V --> W[Rerank (Cross-Encoder hoặc Bi-Encoder tăng cường)]
    W --> X[Top-context được chọn]
end

subgraph Tạo đầu ra
    X --> Y[Mở Prompt builder (templates + quy tắc)]
    Y --> Z[LLM Generator (GPT-4, Claude,...)]
    Z --> O1[Kết quả đầu ra: Danh sách chẩn đoán phân biệt, trích dẫn chứng
    cứ, tiêu chí còn thiếu, gợi ý xét nghiệm]
    O1 --> Clinician
end

subgraph Giám sát & An toàn
    B --> LogDB
    H --> LogDB
    R --> LogDB
    Z --> LogDB
    style LogDB fill:#f8f9fa,stroke:#cccccc
end

```

Mô hình trên thể hiện luồng dữ liệu chính: tài liệu hướng dẫn được chuẩn hóa, chia thành đoạn, sau đó xây cây RAPTOR. Mỗi nút (chi tiết/summarized) được đánh chỉ mục với BM25 và embedding. Khi có truy vấn, hybrid search kết hợp tìm kiếm sparse và semantic ², sau đó dùng cross-encoder để chọn ngữ cảnh phù hợp nhất (ví dụ top-10 mục, ~2000 token như RAPTOR khuyến nghị) ⁸ ⁹. Prompt đầu ra được thiết kế để LLM *không phóng đại*, tập trung vào chứng cứ “có” (bám sát guideline) và gợi ý thêm thông tin khi cần (xét nghiệm bổ sung). Kiến trúc cũng bao gồm các thành phần lưu trữ log/truy vết, API kết nối (ví dụ REST cho retrieval và generation), và cổng “human-in-the-loop” nơi bác sĩ có thể kiểm tra/mở rộng kết quả hoặc can thiệp.

1. Pipeline tiền xử lý chi tiết

(a) Thu thập và chuẩn hóa dữ liệu: Tải toàn bộ các file “Hướng dẫn chẩn đoán và điều trị” từ kcb.vn. Những tài liệu này thường ở PDF, đôi khi scan. Đầu tiên dùng OCR + PyMuPDF/Apache Tika để chuyển sang text, sau đó xác định cấu trúc (mục, tiểu mục, đánh số). Chuyển đổi sang Markdown giúp giữ định dạng (heading, list, bảng) như NICE đã làm ⁵. Ví dụ: phần “1. Chẩn đoán” thành `## 1. Chẩn đoán`, các tiêu đề con thành `### 1.1 <tên>`.

(b) Semantic chunking: Thay vì chia theo số token cố định, áp dụng phương pháp tương tự NICE RAG: **Adaptive split/merge**. Cụ thể, chia theo mục/lục trước. Nếu đoạn con >600 token, tách tiếp (ở đoạn văn/điểm ngắt hợp lý) và *giữ chồng lấp 50 token* với đoạn trước ⁵. Nếu đoạn quá ngắn <200 token, ghép với đoạn liền kề. Cách này đảm bảo mỗi chunk khoảng 300–500 token, có đủ ngữ cảnh và không cắt câu. Kết quả cho ra khoảng 10k–20k chunk (tùy quy mô tài liệu). Thử nghiệm ở NICE thu được 10,195 chunk từ 300 guideline ¹, ta kỳ vọng tương tự cho bộ tài liệu Việt Nam.

(c) **Xây dựng cây RAPTOR:** Dùng RAPTOR để tổ chức thông tin ở nhiều cấp trừu tượng. Các bước:

- **Embedding ban đầu:** Mã hóa mỗi chunk bằng SBERT (Multi-qa-mpnet-base-cos-v1) ⁶. Kết quả các vector 768 chiều.
- **Clustering:** Dùng Gaussian Mixture Model (GMM) để gom nhóm các chunk tương đồng. Để tránh “curse of dimensionality”, giảm chiều trước bằng UMAP ⁷. Tùy chọn các thông số (n_neighbors) để bảo toàn cấu trúc local/global. Dùng tiêu chí BIC để chọn số cluster tối ưu ¹⁰ (kiểm soát phạt độ phức tạp và độ phù hợp). Quá trình có thể chạy đệ quy: nếu một cluster rất lớn vượt giới hạn token (ví dụ > 5000 token), tách cluster đó tiếp.
- **Summarization:** Với mỗi cluster, gọi LLM (ví dụ GPT-3.5-turbo) để tạo ra **tóm tắt** nội dung cluster. Prompt ví dụ: “Tóm tắt các mục quan trọng trong đoạn văn sau” + gán list các chunk. Kết quả là văn bản ngắn hơn (nút cha). Lặp lại embedding GMM trên các summary này để hình thành các lớp cao hơn. Quá trình này dừng khi không thể chia thêm (mỗi node cuối cùng chứa 1 cluster).
- **Overlap/Oversight:** Đảm bảo khi tóm tắt, vẫn giữ các chi tiết quan trọng bằng cách chồng lấp thông tin: ví dụ cho phép 1 chunk xuất hiện trong 2 cluster khi có thể (soft clustering như GMM). Điều này giúp thông tin quan trọng xuất hiện ở nhiều nhánh, tăng khả năng tìm đúng ngữ cảnh.

Kết quả: một cây đa cấp từ **chi tiết đến khái quát**. Nút lá là text nguyên bản, các lớp trên là summary trừu tượng hơn. Dữ liệu bổ sung (metadata) như nguồn file, mục hướng dẫn, phần mục lục ban đầu, được lưu kèm để provenance.

2. Đánh chỉ mục và truy xuất

(a) **BM25 (Sparse Index):** Sử dụng Okapi BM25 (plugin như Elasticsearch hoặc Whoosh) với nội dung chunks gốc. Cấu hình tương tự NICE: token hóa *lemmatize* (theo đại từ, gốc từ) ¹¹, hệ số $k1 \approx 1.7$, $b \approx 0.83$ (nên điều chỉnh thử nghiệm theo độ dài trung bình text Việt). BM25 mạnh ở việc khớp các thuật ngữ y tế đặc thù (ví dụ “viêm đa khớp dạng thấp”), vốn rất quan trọng trong chẩn đoán lâm sàng.

(b) **Embedding (Dense Index):** Đối với mỗi node (lá và summary), tính embedding SBERT. Ta lưu vào một vector store (ví dụ sử dụng FAISS). Có thể lựa chọn mô hình embedding độ lớn khác nhau tùy ngân sách:

- *Low-end:* sentence-transformer nhỏ (768 chiều, xử lý CPU/GPU vừa phải).
- *Mid-range:* dense biến thể DPR hay OpenAI text-embedding-3-large (1536 chiều, yêu cầu GPU).
- *High-end:* các embeddings phức tạp hơn (như Voyager hay Qwen3) nếu có license. Bảng so sánh (ví dụ):

Giải pháp embedding	Dạng	Kích thước vector	Ưu điểm	Nhược điểm
SBERT/multi-qa-mpnet	Dense (768d)	768	Open-source, rẻ, chạy tốt cho search đa ngôn ngữ	Ít chiều hơn, đôi khi kém chính xác so với mô hình lớn hơn
OpenAI Text-Embedding-3-large	Dense (1536d)	1536	Kết quả thử nghiệm cao (NICE dùng Voyage ~1024d) ¹²	Chi phí API cao, cần GPU

Giải pháp embedding	Dạng	Kích thước vector	Ưu điểm	Nhược điểm
BM25	Sparse	–	Nhanh, không cần GPU, tốt với thuật ngữ y khoa	Không hiểu nghĩa ngữ cảnh, không bắt synonyms
Hybrid (BM25 + Dense)	–	–	Tận dụng ưu điểm cả hai; recall cao (NICE đạt 99.1%@10) ¹²	Phức tạp, cần điều chỉnh trọng số fusion

(c) Đánh chỉ mục: Lưu tất cả node vào cả hai hệ thống: 1 chỉ mục BM25 và 1 cơ sở vector (FAISS hoặc dịch vụ tương tự). Mỗi entry gắn thông tin node-id, layer (chi tiết hay summary), nguồn guideline và phân đoạn văn bản. Điều này giúp truy xuất nhanh và cũng hỗ trợ kê gần offset trích dẫn cho LLM.

(d) Truy xuất hỗn hợp (Hybrid Retrieval): Khi có truy vấn triệu chứng từ người dùng, hệ thống thực hiện:

1. Tiền xử lý truy vấn (chuyển đồng nghĩa, chuẩn hóa biểu tượng, bỏ dấu câu) và tạo embedding.
2. **Truy vấn BM25:** tìm top-N (ví dụ N=20) đoạn gốc phù hợp với từ khóa (hạn chế stopwords y học phổ thông).
3. **Truy vấn dense:** tính cosine giữa vector query và tất cả vector node (leaf + summary). Lấy top-M (M≥20).
4. **Hợp nhất kết quả (WRRF):** Dùng công thức *Weighted RRF* để kết hợp thứ hạng BM25 và dense ² :

$$\text{Score}(\text{doc}) = \sum_{m \in \{\text{BM25}, \text{dense}\}} \frac{w_m}{k + \text{rank}_m(\text{doc})}$$

với trọng số $w_{\text{BM25}}, w_{\text{dense}}$ được tinh chỉnh (NICE khuyến nghị $w_{\text{BM25}}=1, w_{\text{dense}}=5, k=40$ ²). Kết quả là một bảng xếp hạng nhị phân, ưu tiên các đoạn xuất hiện cao trong cả hai phương pháp.

1. **Reranking cross-encoder:** Trình tự hybrid đưa ra top-k (ví dụ 10-15) đoạn. Tiếp theo, đưa từng cặp (câu hỏi + từng đoạn) vào mô hình Cross-Encoder (ví dụ BERT làm reranker đã huấn luyện trên QA hoặc sử dụng GPT-4 dưới dạng zero-shot rank) để tính độ tương quan ngữ cảnh sâu hơn. Cross-Encoder sẽ đánh giá trực tiếp sự liên quan (quá trình này chậm hơn nhưng chính xác hơn). Kết quả cuối là top-k đoạn được sắp xếp chuẩn xác.

(e) Collapsed-tree Retrieval: Phương pháp truy vấn đã thống nhất trong RAPTOR là “collapsed-tree” ¹³ ¹⁴ . Thay vì đi từ gốc theo tầng, ta **nén toàn bộ cây thành một danh sách node**. Khi truy vấn, đánh giá cosine query với mọi node, rồi chọn lần lượt các node có điểm cao nhất **đến khi tổng token ≤ giới hạn** (ví dụ ~2000 token) ¹⁵ ¹⁶ . Cách này linh hoạt vì có thể chọn cả summary và chi tiết phù hợp mức độ granularity câu hỏi. Kết quả RAPTOR cho thấy collapsed tree luôn tốt hơn traversal (như hình minh họa) ¹⁴ . Ví dụ, dùng token-limit 2000 (~top-20 nodes) thường tối ưu.

3. Tích hợp với LLM và Prompt Engineering

(a) Prompt templates: Dạng đầu ra mong muốn là “chẩn đoán phân biệt dựa trên triệu chứng” có cấu trúc. Prompt có thể giống mẫu sau (trong tiếng Việt):

Bạn là một trợ lý y khoa. Người dùng cung cấp danh sách triệu chứng/số liệu xét nghiệm. Hãy đưa ra:

1. ****Danh sách các bệnh/điều kiện nghi ngờ**** (có thứ tự ưu tiên), mỗi bệnh kèm ****chứng cứ từ hướng dẫn**** (đoạn văn trích dẫn + mã) hỗ trợ.
 2. ****Tiêu chí còn thiếu**** cho mỗi chẩn đoán (ví dụ: Xét nghiệm hay triệu chứng chưa có) để xác định chính xác.
 3. ****Đề xuất xét nghiệm hoặc bước tiếp theo**** để làm rõ chẩn đoán.
 4. ****Không phóng đại****, chỉ dựa vào ngữ cảnh đã đưa. Nếu không đủ thông tin trong hướng dẫn, hãy yêu cầu thêm thông tin.
- Định dạng trả lời: Markdown với danh sách số và đầu mục rõ ràng.

Guardrails:

- **Không bịa thông tin:** Luôn trích dẫn chính xác nguồn (ví dụ "(Hướng dẫn CĐ-ĐT CXK 2014, mục 5.3)").
- **Nếu không có kết quả:** Nếu ngữ cảnh lấy được không đủ để kết luận bệnh nào, trả lời dạng: "Không tìm thấy thông tin rõ ràng trong hướng dẫn để xác định" ¹⁷.
- **Không hỏi độc lập:** Trả lời chỉ sau khi thu thập đủ context, tránh đoán bừa.
- **Chèn quy tắc an toàn:** Ví dụ, nếu triệu chứng nguy hiểm (suy hô hấp, mất ý thức), chèn cảnh báo (cấp cứu) và dừng trả lời.

Ví dụ đầu vào (query) đa triệu chứng tiếng Việt:

"Nam 56 tuổi, đau ngực trái dữ dội lan vai, khó thở, vã mồ hôi. Huyết áp 160/100, nhịp tim 110, ECG cho thấy ST chênh lên, men tim tăng cao."

Ví dụ đầu ra kỳ vọng:

1. ****Nhồi máu cơ tim cấp (NMCT):**** Triệu chứng ngực trái lan vai, vã mồ hôi là điển hình; hướng dẫn NMCT của Bộ Y tế 2014 (mục 2.1) nói triệu chứng như vậy. Xét nghiệm men tim dương tính hỗ trợ.
 - ***Chứng cứ:** "...đau ngực lan vai, đổ mồ hôi..." (Hướng dẫn NMCT, trang 5) ^{【来源 1】}.
 - ***Tiêu chí chưa có:** Cần xem hình ảnh tim (siêu âm, CATH) để chắc chắn, đánh giá chỉ điểm sinh học.
 - ***Xét nghiệm gợi ý:** Siêu âm tim khẩn cấp; chỉ số BNP/CK-MB thêm.
2. ****Cơn đau thắt ngực không ổn định:**** Triệu chứng gần giống NMCT nhưng men tim chưa chẩn đoán NMCT; cần ECG và test gắng sức.
 - ***Chứng cứ:** Hướng dẫn CĐ-ĐT cơn đau thắt ngực (BXH 2016) (mục 4.2) nêu: đau ngực khi gắng sức, giảm thiểu khi nghỉ.
 - ***Chưa có:** Cơn đau chưa được nghỉ/nghỉ ngơi có giảm đau chưa.
 - ***Gợi ý:** Test gắng sức/CT mạch vành.
3. ****Viêm màng ngoài tim:**** Đau ngực có thể do viêm màng ngoài; cần hỏi triệu chứng giảm khi ngồi lên.
 - ***Chứng cứ:** Hướng dẫn cơ xương khớp (2014, mục 7.1) mô tả viêm màng ngoài:

đau ngực tăng khi nằm.

- *Chưa có:* Dấu hiệu giảm đau khi ngồi.
- *Gợi ý:* Khám màng phổi, siêu âm tim kiểm tra dịch màng ngoài.

Kết quả là *structure*: danh sách phân biệt kèm *Bảng chứng* (trích trực tiếp), *Tiêu chí thiếu*, *Xét nghiệm đề xuất*.

Đây là dạng chuẩn như yêu cầu kiểm soát trả lời y khoa, giúp bác sĩ đối chiếu.

4. Đánh giá và phép đo hiệu suất

Metrics Retrieval

- **Recall@k, MRR (Mean Reciprocal Rank), nDCG:** Đo khả năng lấy ra đoạn hướng dẫn đúng trong top-k. Ví dụ NICE RAG đạt Recall@1=81%, Recall@10=99.1%, MRR≈0.814¹. Mục tiêu tương tự: tỷ lệ correct chunk ở top k >= 95%, MRR cao (~0.8+). Sử dụng tập câu hỏi truy xuất từ hướng dẫn Việt (tạo thủ công hoặc tổng hợp).
- **Latency:** Thời gian từ query đến trả kết quả retrieval nên nằm trong phạm vi 1–2 giây cho tìm kiếm (trên phần cứng phù hợp).

Metrics Generation

- **Precision/Recall câu trả lời:** Đánh giá đầu ra của LLM so với “đáp án chuẩn” (có thể từ bác sĩ). Dùng đo lường NLP như BLEU, ROUGE nhưng quan trọng hơn **FactScore** (độ trung thực)¹⁸ và **MED-F1** (độ bao phủ entity y tế). Có thể tạo tập test gồm các câu hỏi lâm sàng tiêu chuẩn và câu trả lời mẫu (tin cậy) để tính.
- **Clinical F1 (độ chính xác y khoa):** Đo bao nhiêu ý định lâm sàng (chẩn đoán/tiêu chí) được liệt kê đúng. Ví dụ coi mỗi chẩn đoán đúng là “true positive” nếu có bằng chứng.
- **Human eval:** Bác sĩ đánh giá chất lượng đầu ra (đúng – sai – đủ chi tiết, có hallucinate không). Tỷ lệ đồng thuận cao trên các cases tiêu chuẩn. Cũng có thể đo lường *confidence score* nếu dùng LLM ra xác suất hoặc điểm.

Thiết lập thử nghiệm

- **Dữ liệu kiểm thử:** Xây các trường hợp câu hỏi (triệu chứng bệnh) thực tế và giả lập, kèm đáp án tham khảo. Kết hợp: câu hỏi rút ra từ guideline có sẵn, bộ cas bệnh mẫu, hoặc tạo tình huống phức hợp để kiểm thử multi-hop reasoning (như trường hợp QuALITY, NarrativeQA trên văn bản dài)¹⁹.
- **So sánh A/B:** Đối chiếu: (1) RAG+RAPTOR vs (2) chỉ LLM không truy xuất, vs (3) RAG thuần (không cây). Đánh giá sự khác biệt về faithfulness (tỷ lệ hallucinate), độ chính xác y tế và tốc độ. NICE RAG chứng minh RAG tăng faithfulness rất mạnh (từ 35% lên ~99.5%)²⁰; RAPTOR tăng nhiều trên các QA phức tạp¹⁹.
- **Continuous evaluation:** Giám sát recall@k và độ chính xác đầu ra định kỳ. Ví dụ, mỗi tháng đánh giá lại bộ câu hỏi mẫu để phát hiện drift. Phát hiện sớm nếu metric tụt (đổi guideline, lỗi indexing). Lập dashboard theo dõi: số truy vấn, thời gian, tỷ lệ trả “không tìm thấy”, feedback user.

5. Phần cổng an toàn và tuân thủ

- **Gate kiểm duyệt (rule-based):** Trước khi trả lời, hệ thống kiểm tra triệu chứng nguy hiểm (theo danh sách cảnh báo). Nếu query chứa tín hiệu cấp cứu (ví dụ “ngất”, “mất ý thức”), prompt cảnh báo bác sĩ cấp cứu ngay.
- **Không PHI:** Thiết kế giao diện yêu cầu người dùng không nhập thông tin cá nhân (họ tên, mã BHYT). Tất cả log query ẩn danh ngay khi nhận vào. Mỗi bản ghi log chỉ lưu triệu chứng chung, không lưu thông tin định danh.
- **Bảo mật dữ liệu:** Kiến trúc hướng dẫn (công bố) thì không nhạy cảm, nhưng lưu ý rằng vector embeddings và logs truy vấn có thể lạm dụng. Theo hướng dẫn bảo mật, lưu vector và logs trong hệ thống bảo vệ (cơ sở trên-premises hoặc đám mây tuân thủ ISO 27001, GDPR/HIPAA). Dữ liệu truyền (API calls, embedding) mã hóa SSL.
- **Quyền truy cập:** Chỉ admin/nhân viên y tế mới quản lý hệ thống. Bác sĩ truy cập UI, không truy cập trực tiếp cơ sở dữ liệu. Thiết lập audit trail: mọi truy vấn và kết quả đều ghi log kèm user-id (ẩn danh nếu cần) để sau này kiểm tra nguồn gốc thông tin (provenance).
- **Các biện pháp riêng:** Cần xem xét *privacy risk* trong RAG y tế. Các nghiên cứu chỉ ra khả năng *inversion attack* trên embedding có thể lộ thông tin nhạy cảm ²¹. Đề xuất:
 - Chạy embedding và retriever nội bộ (on-prem) để dữ liệu nhạy không thoát ra dịch vụ bên ngoài.
 - Trước khi xử lý, áp dụng *anonymization* trên query (lọc thông tin cá nhân) ²¹.
 - Cân nhắc differential privacy cho vector lưu trữ nếu cần (cơ chế Gauss noise) ²¹.
 - **Tuân thủ quy định:** Hệ thống cần chứng minh khả năng bảo mật (ví dụ báo cáo internal audits, thông báo cho cơ quan quản lý y tế) để đáp ứng Nghị định riêng lẻ về bảo mật thông tin y tế.

6. Độ trễ, khả năng mở rộng và triển khai

- **Latency/Throughput:** Mục tiêu thời gian phản hồi (không bao gồm thao tác của bác sĩ) khoảng 1-2 giây cho retrieval (BM25+FAISS), ~0.5-1 giây cho reranking, ~1-3 giây cho LLM generation (tùy model). Tổng độ trễ dưới 5s chấp nhận được cho truy vấn chẩn đoán. Để đạt được:
 - Chạy BM25 trên cluster Elasticsearch (CPU).
 - Lưu embedding trong FAISS (GPU/CPU) sử dụng HNSW hoặc PQ để truy xuất nhanh.
 - Reranker Cross-Encoder trên GPU (batch 10).
- LLM: có thể dùng GPT-4-32k (đám mây) hoặc một LLM nhỏ tự host (Ví dụ LLaMA-3) nếu cần on-prem.
- **Khả năng mở rộng:**
 - *Mức thấp:* Ít tài nguyên, chỉ BM25 + off-the-shelf SBERT, LLM cloud. Được dùng cho POC, prototype.
 - *Mức trung bình:* VM/GPU on-prem, chạy SBERT có GPU, sử dụng OpenAI API cho GPT-4 (chi phí trả theo usage). Để xử lý khối văn bản guideline lớn (vài chục GB).

- **Mức cao:** Toàn bộ on-prem (FAISS, LLM local), cài Container/K8s để scale-out. Lưu ý NCR: For ultra-low latency, dùng vector DB hỗ trợ hybrid search nhúng GPU (like NVIDIA cfaiss) hoặc cloud vector DB (Pinecone, Weaviate).
- **Triển khai:** Nên thiết kế theo microservices: *Retriever service* (BM25 server + FAISS service + API), *Reranker service*, *Generator service*, *Prompt service*. Giao tiếp qua REST/gRPC. Cho phép thay thế từng module (ví dụ thay OpenAI bằng LLM khác). Lưu trữ: file guideline raw, indices, vector DB. Logging/Audit riêng biệt.

7. Kế hoạch đánh giá và cập nhật liên tục

- **Dữ liệu kiểm thử:** Xây tập mẫu câu hỏi y khoa Việt (có/không đầy đủ triệu chứng) kèm đáp án chuẩn do chuyên gia chỉ định. Có thể tái sử dụng các ca lâm sàng đã công bố hoặc dịch/gợi ý các trường hợp ví dụ từ sách y khoa. Hỗn hợp câu hỏi đơn (hỏi 1 triệu chứng đặc trưng) và đa-hop (kết hợp nhiều triệu chứng cần so sánh). Ví dụ tương tự QuALITY (5000 token) hoặc LongAnswer.
- **Đánh giá Retrieval:**
 - Chạy truy vấn truy xuất từ tập test; tính Recall@1, Recall@5, MRR. Phấn đấu Recall@10 $\geq$ 95%.
 - Tạo “Ground-truth” cho mỗi truy vấn: các đoạn guideline thực sự chứa thông tin chẩn đoán.
- **Đánh giá Generation:**
 - Dùng BLEU/ROUGE để so sánh câu trả lời với câu mẫu (chỉ mang tính tham khảo).
 - Tính **FactScore** (độ trung thực với nguồn) ¹⁸: kiểm tra tỷ lệ thông tin trong trả lời có tồn tại trong context.
 - **MED-F1:** F1 trên các thực thể y tế (bệnh, thuốc) được nhắc tới.
 - Quan trọng nhất: đánh giá bởi bác sĩ (Adjudication). Mỗi câu trả lời được 2-3 chuyên gia chấm (đúng/sai, đủ/chưa, có hallucination không, etc). Tính điểm trung bình, thống kê agreement.
- **A/B Testing và Monitoring:**
 - So sánh hai phiên bản hệ thống (vd. tắt bật RAPTOR tree hoặc dùng dense model khác) qua metric retrieval & generation trên cùng tập thử.
 - Triển khai beta cho một nhóm bác sĩ và thu thập feedback: đo độ chính xác, tỉ lệ sử dụng hệ thống (user satisfaction).
 - Giám sát sau khi vận hành: log lịch sử truy vấn và đầu ra; nếu xuất hiện nhiều “không tìm thấy” hoặc câu trả lời sai, xem xét điều chỉnh lại index hay retrain module.
- **Cập nhật kiến thức:** Hướng dẫn y khoa thay đổi theo năm. Xác định chu kỳ rebuild cây: ví dụ mỗi năm một lần khi Bộ Y tế ra quyết định mới. Ngoài ra, nếu hướng dẫn có supplement, tự động thêm vào chỉ mục và tái chạy quá trình chunking+embedding cho nội dung mới.

- Lên kế hoạch tái xây RAPTOR (clustering) định kỳ (6-12 tháng) hoặc khi nội dung mới vượt 10% corpus.
- Tự động cập nhật chỉ mục BM25 và vector DB mỗi lần thêm/chỉnh sửa documents (chi tiết tùy công cụ indexing).

8. Ví dụ minh họa

Ví dụ truy vấn: “Bệnh nhân nữ 45 tuổi, đau khớp ngón tay, cứng buổi sáng kéo dài 1 giờ, xét nghiệm RF(+), anti-CCP(+). Hỏi bệnh gì? Nếu chưa đủ, cần kiểm tra gì thêm?”

Mẫu đầu ra mong muốn:

- **Viêm đa khớp dạng thấp (RA):****
 - ***Chứng cứ:** Triệu chứng đau khớp ngón, cứng buổi sáng >30 phút, RF(+), Anti-CCP(+) hoàn toàn khớp với tiêu chuẩn ACR/EULAR cho RA. (Theo Hướng dẫn CXK 2014, mục 3.2: “Đau khớp đối xứng, cứng buổi sáng, kháng thể anti-CCP”).
 - ***Tiêu chí thiếu:** Chưa có xét nghiệm CRP/ESR để đánh giá viêm. Chưa rõ có tổn thương X-quang khớp (gãy vôi hoá).
 - ***Gợi ý thêm:** Làm X-quang bàn tay để đánh giá hủy xương. Xét nghiệm CRP/ESR để đánh giá mức độ viêm, theo dõi đáp ứng.
- **Lupus ban đỏ hệ thống (SLE):****
 - ***Chứng cứ:** Một số triệu chứng có thể trùng (đau khớp, kháng thể), nhưng thường RA chủ yếu, SLE cần thêm dấu hiệu khác (phát ban hình cánh bướm,...).
 - ***Thiếu:** Cần test ANA/anti-dsDNA, triệu chứng ngoài khớp (ban, tổn thương thận).
 - ***Gợi ý:** Xét nghiệm ANA, đánh giá đa ngành.
- **Vôi hoá khớp (Osteoarthritis dạng hiếm):****
 - ***Chứng cứ:** Ít hợp lý vì RA đã thuyết phục hơn, nhưng luôn cân nhắc nếu bệnh nhân >50 tuổi.
 - ***Thiếu:** Hình ảnh học (hẹp khe khớp, gai xương) để xác nhận.
 - ***Gợi ý:** Chụp X-quang kiểm tra.

Kết quả liệt kê khoa học như trên giúp người dùng nhanh nhận ra khả năng chính (RA là mục 1), các bước cần kiểm tra thêm (CRP, X-quang), đồng thời cảnh báo “không bịa” thông tin mới. Mỗi điểm đều trích nguồn hướng dẫn cụ thể (thể hiện qua mã trang hay mục) để tăng độ tin cậy.

9. Bảng so sánh thiết kế chính

Embedding & Retrieval Engines:

Thành phần	Lựa chọn	Ưu điểm	Nhược điểm
Sparse (BM25)	Elasticsearch/Whoosh	Nhanh, tiết kiệm, tốt với từ khóa y học đặc thù	Không capture ngữ nghĩa, cần tuning (k1,b) ¹¹

Thành phần	Lựa chọn	Ưu điểm	Nhược điểm
Dense (SBERT, DPR)	Mô hình embedding (768–1536 chiều)	Hiểu nghĩa (synonym), cải thiện recall, chuyên biệt nếu fine-tune	Tốn GPU, phải tạo index vector, latency cao hơn khi tìm kiếm
Hybrid (Weighted RRF)	BM25 + Dense (wr=5:1) ²	Kết hợp ưu điểm, recall đạt 99% tại top 10 ²²	Cần compute gấp đôi, phức tạp cài đặt, cần điều chỉnh trọng số
FAISS vs Cloud Vector DB	FAISS (on-prem) vs Pinecone, Weaviate	FAISS offline, không tốn phí; Cloud DB dễ tích hợp, scale tự động	FAISS cần scale manual; Cloud DB phụ thuộc mạng, tính phí

Reranker & Generation:

Thành phần	Lựa chọn	Ưu điểm	Nhược điểm
Cross-Encoder (BERT)	BERT/BioBERT fine-tune	Rerank chính xác, hiểu mối quan hệ query-doc	Chậm (O(k)), cần GPU, chi phí lớn
Bi-Encoder (DPR)	DPR hoặc SBERT	Nhanh cho retrieval (precomputed), nhẹ	Ít chính xác hơn, chỉ nhìn riêng lẻ
LLM (GPT, Claude)	GPT-4,...	Kết hợp tốt prompt, ngôn ngữ tự nhiên đẹp	Chi phí cao, khó kiểm soát tính nhân văn và hallucination
Generator	GPT-4 (đám mây)	Kết quả chính xác cao, có cảm giác mượt mà	Chịu phí theo token, phụ thuộc mạng
Generator	LLaMA-3/ MedLM (host)	Chủ động hơn, tùy chỉnh model	Cần build tốt prompt, có thể yếu hơn GPT-4

Triển khai:

Môi trường	Cách triển khai	Ưu điểm	Nhược điểm
On-premise	Cài toàn bộ (FAISS, local LLM, ES, GPU Server)	Bảo mật cao, đáp ứng nhanh (no internet latency)	Chi phí đầu tư hạ tầng, bảo trì nhân lực
Cloud (API)	FAISS trên VM + dịch vụ LLM (OpenAI), Elastic Cloud	Khởi động nhanh, dễ mở rộng	Tốc độ phụ thuộc mạng, tính phí API, cần đảm bảo PHI qua (encryption, compliance)
Hybrid	On-prem index + remote LLM	Giảm chi phí hạ tầng (LLM chia sẻ), vẫn kín dữ liệu	Quản lý phức tạp, phải bảo vệ data transfer

10. Kết luận

Bằng việc kết hợp **Hybrid Retrieval (BM25 + Dense)** từ NICE-RAG ² và **RAPTOR Tree với Collapsed Search** ^{13 14}, hệ thống mới sẽ vừa “bắt” được các triệu chứng đặc trưng qua từ khóa chuyên ngành, vừa nắm được **toàn cảnh** hướng dẫn lâm sàng. Thiết kế này giảm thiểu hallucination (đã được chứng minh tăng faithfulness lên ~99% với RAG ²⁰) và hỗ trợ reasoning nhiều bước. Quy trình triển khai chi tiết (từ ingestion, chunking, lập tree, indexing, đến prompt cụ thể) đảm bảo pipeline sẵn sàng sản xuất: modular, có thể scale và kiểm soát tốt tính an toàn ^{18 21}.

Các đề xuất trong báo cáo này (hệ thống tính toán, so sánh kỹ thuật, prompt mẫu, pipeline đánh giá) xây dựng trên các nguồn nghiên cứu hàng đầu (NICE RAG và RAPTOR) và áp dụng cho ngữ cảnh tài liệu y tế Việt Nam. Chúng tôi có thể hỗ trợ thêm bằng sơ đồ chi tiết (SVG), flowchart Mermaid, và hướng dẫn triển khai cụ thể (cấu hình index, code mẫu) nếu cần.

Nguồn tham khảo: Thiết kế dựa trên nghiên cứu NICE-RAG ^{1 2}, phương pháp RAPTOR ^{7 13}, và các chỉ dẫn review RAG Y tế ^{18 21}, cùng kinh nghiệm phát triển hệ thống tương tự.

^{1 2 5 11 12 17 20 22} [2510.02967] Grounding Large Language Models in Clinical Evidence: A Retrieval-Augmented Generation System for Querying UK NICE Clinical Guidelines

<https://ar5iv.labs.arxiv.org/html/2510.02967v1>

^{3 4 6 7 8 9 10 13 14 15 16 19} [2401.18059] RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval

<https://ar5iv.labs.arxiv.org/html/2401.18059v1>

¹⁸ Retrieval-Augmented Generation (RAG) in Healthcare: A Comprehensive Review – DOAJ

<https://doaj.org/article/84f6b26923524b2e92b8a3655527a9fa>

²¹ Privacy Challenges and Solutions in RAG-Enhanced LLMs for Healthcare Chatbots: A Review of Applications, Risks, and Future Directions

<https://arxiv.org/html/2511.11347v2>